Stat 6559 / 4560; DSCI 6659: Assignment #3

Zahra Jabari

2024-04-04

Question 1.Construct the Hotelling T^2 charts for future observations using the a simulated data Simulation Set-up
a) Use the set.seed("6659")
b) Generate 100 observations from bivariate normal distribution with $\mu = (2, 5)$ and covariance matrix $\text{var}(x_1) = 1$; $\text{var}(x_2) = 0.5$, $\text{cov}(x_1, x_2) = 0.3$.
Answer:

```
suppressWarnings({library(MASS)})
set.seed("6659")
n<-100
mu<-c(2,5)
sigma<-matrix(c(1,0.3,0.3,0.5),2,2)
x<-mvrnorm(n,mu,sigma)
head(x)
```

```
##          [,1]     [,2]
## [1,] 2.387119 4.267006
## [2,] 1.499730 4.151931
## [3,] 3.259721 4.984143
## [4,] 2.663256 4.801240
## [5,] 3.356648 5.139510
## [6,] 1.493616 5.160085
```

```
tail(x)
```

```
##            [,1]     [,2]
##  [95,] 1.344467 5.481071
##  [96,] 1.888385 5.013397
##  [97,] 2.395742 5.503529
##  [98,] 3.587228 4.880884
##  [99,] 3.038306 4.556231
## [100,] 3.303515 4.537516
```

c) Estimate the classical estimators of mean and covariances.
Answer:

```
Mu_hat_Classic<-colMeans(x)
S_hat_Classic<-cov(x)
cat("\n classical approach\n")
```

```
##
##  classical approach
```

```
Mu_hat_Classic
```

```
## [1] 2.262580 4.921275
```

```
S_hat_Classic
```

```
##           [,1]     [,2]
## [1,] 0.8909968 0.294199
## [2,] 0.2941990 0.450607
```

```
suppressWarnings({library(rrcov)})
suppressWarnings({library(robustbase)})

estim_robust_MCD<-covMcd(x,cor=FALSE)
Mu_hat_Robust_MCD<-estim_robust_MCD$center
S_hat_Robust_MCD<-estim_robust_MCD$cov

estim_robust_MVE<-CovMve(x)
Mu_hat_Robust_MVE<-estim_robust_MVE$center
S_hat_Robust_MVE<-estim_robust_MVE$cov

cat("\n Classical estimators of mean and covariances \n")
```

```
##
##  Classical estimators of mean and covariances
```

```
Mu_hat_Classic
```

```
## [1] 2.262580 4.921275
```

```
S_hat_Classic
```

```
##           [,1]     [,2]
## [1,] 0.8909968 0.294199
## [2,] 0.2941990 0.450607
```

```
cat("\n Robust(MCD) estimators of mean and covariances \n")
```

```
##
##  Robust(MCD) estimators of mean and covariances
```

```
Mu_hat_Robust_MCD
```

```
## [1] 2.331226 4.951357
```

```
S_hat_Robust_MCD
```

```
##           [,1]      [,2]
```

```
## [1,] 0.9170622 0.2698410
## [2,] 0.2698410 0.3922925
```

```
cat("\n Robust(MVE) estimators of mean and covariances \n")
```

```
##
##  Robust(MVE) estimators of mean and covariances
```

```
Mu_hat_Robust_MVE
```

```
## [1] 2.322748 4.993813
```

```
S_hat_Robust_MVE
```

```
##           [,1]      [,2]
## [1,] 0.9009276 0.2590125
## [2,] 0.2590125 0.4641337
```

d) Generate 25 future observations, using bivariate normal distribution with $\mu = (2, 5)$ and covariance matrix $\text{var}(x_1) = 1$; $\text{var}(x_2) = 0.5$, $\text{cov}(x_1, x_2) = 0.3$.
Answer:

```
set.seed("6659")
n_new<-25
mu_new<-c(2,5)
sigma_new<-matrix(c(1,0.3,0.3,0.5),2,2)
x_new<-mvrnorm(n_new,mu_new,sigma_new)
x_new
```

```
##             [,1]     [,2]
##  [1,] 2.07189984 4.940013
##  [2,] 1.25007705 4.684950
##  [3,] 3.03016694 5.474249
##  [4,] 2.40028971 5.362684
##  [5,] 2.78587400 6.358136
##  [6,] 1.99468847 4.090274
##  [7,] 1.20204564 4.616827
##  [8,] 2.39197736 6.082168
##  [9,] 0.76032031 5.303617
## [10,] 2.25447662 4.942188
## [11,] 4.22214535 4.862009
## [12,] 3.02031050 4.363667
## [13,] 3.41564995 6.242286
## [14,] 2.57403577 4.969157
## [15,] 0.93777437 4.260924
## [16,] 2.95558462 4.370191
## [17,] 1.20345032 5.336655
## [18,] 1.70918504 5.164499
## [19,] 0.01080933 4.251975
## [20,] 1.81037495 4.986103
## [21,] 2.04721023 6.083392
## [22,] 2.57992800 5.366280
## [23,] 1.57504767 5.509975
## [24,] 3.17041171 4.838395
## [25,] 2.27233398 5.797148
```

e) Draw three T^2 control chart for future observation using classical estimator and robust estimators of mean and covariance matrix. Draw your conclusions.

Answer:

```
Stats_T2_func<-function(x,x_bar,S,m,n,p,conf_level){
statistics<-numeric(nrow(x))
for (i in 1:nrow(x)){
statistics[i]<-t(x[i,]-x_bar)%*%solve(S)%*%(x[i,]-x_bar)
}
UCL<-((p*(m-1))/(m-p))*qf(conf_level,df1=p,df2=m-p,ncp=0,
                          lower.tail=TRUE,log.p = FALSE)
return(list(statistics_T2=statistics,LCL=0,UCL=UCL))
}
#-----------------------------------
#         classic
#-----------------------------------
Mu_hat_Classic<-colMeans(x)
S_hat_Classic<-cov(x)
result_classic<-Stats_T2_func(x_new,Mu_hat_Classic,S_hat_Classic,25,1,2,0.95)
index_classic_UCL<-which(result_classic$statistics_T2>result_classic$UCL)
result_classic_UCL<-result_classic$statistics_T2[index_classic_UCL]
index_classic_LCL<-which(result_classic$statistics_T2<result_classic$LCL)
result_classic_LCL<-result_classic$statistics_T2[index_classic_LCL]
#-----------------------------------
#         Robust(MCD)
#-----------------------------------
estim_robust_MCD<-covMcd(x,cor=FALSE)
Mu_hat_Robust_MCD<-estim_robust_MCD$center
S_hat_Robust_MCD<-estim_robust_MCD$cov

result_robust_MCD<-Stats_T2_func(x_new,Mu_hat_Robust_MCD,S_hat_Robust_MCD,25,1,2,0.95)
index_robust_MCD_UCL<-which(result_robust_MCD$statistics_T2>result_robust_MCD$UCL)
result_robust_MCD_UCL<-result_robust_MCD$statistics_T2[index_robust_MCD_UCL]
index_robust_MCD_LCL<-which(result_robust_MCD$statistics_T2<result_robust_MCD$LCL)
result_robust_MCD_LCL<-result_robust_MCD$statistics_T2[index_robust_MCD_LCL]

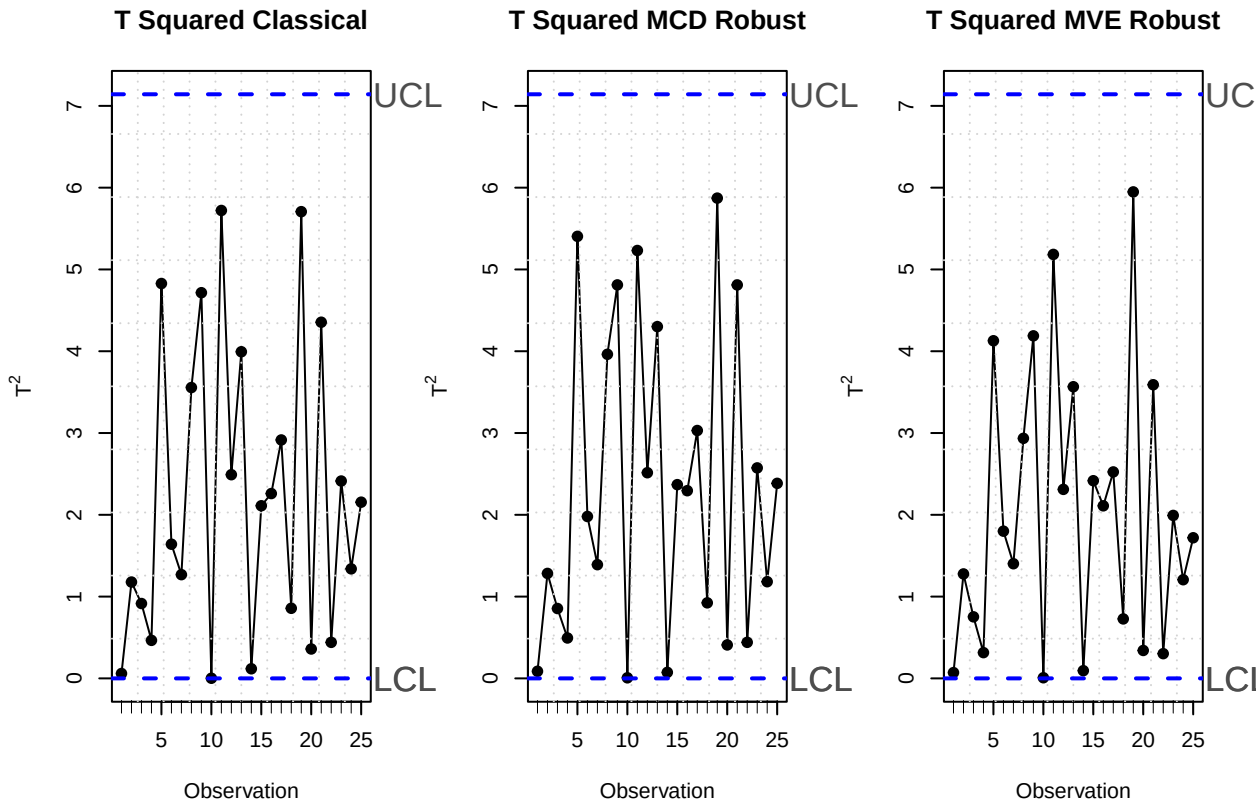
#-----------------------------------
#         Robust(MVE)
#-----------------------------------
estim_robust_MVE<-CovMve(x)
Mu_hat_Robust_MVE<-estim_robust_MVE$center
S_hat_Robust_MVE<-estim_robust_MVE$cov

result_robust_MVE<-Stats_T2_func(x_new,Mu_hat_Robust_MVE,S_hat_Robust_MVE,25,1,2,0.95)
index_robust_MVE_UCL<-which(result_robust_MVE$statistics_T2>result_robust_MVE$UCL)
result_robust_MVE_UCL<-result_robust_MVE$statistics_T2[index_robust_MVE_UCL]
index_robust_MVE_LCL<-which(result_robust_MVE$statistics_T2<result_robust_MVE$LCL)
result_robust_MVE_LCL<-result_robust_MVE$statistics_T2[index_robust_MVE_LCL]
#-------------------------------------------------------------------------------
par(mfrow=c(1,3))
plot(1:25,result_classic$statistics_T2,type="l",ylab=expression(T^2),xlab="Observation",
     ylim=c(result_classic$LCL,result_classic$UCL),
     main="T Squared Classical")
grid(nx=10,ny=10,lwd=1,lty=3,equilogs=F)
```

```
points(1:25,result_classic$statistics_T2,type="p",pch=20,cex=1.5)
abline(h=result_classic$UCL,lty=2,col="blue",lwd=2)
abline(h=result_classic$LCL,lty=2,col="blue",lwd=2)
mtext("UCL",side=4,at=c(25,result_classic$UCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
mtext("LCL",side=4,at=c(0,result_classic$LCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
points(index_classic_LCL,result_classic_LCL,col="red",type="p",pch=20)
points(index_classic_UCL,result_classic_UCL,col="red",type="p",pch=20)
xmintick=seq(1,25,by=1)
axis(1,at=xmintick,labels=FALSE,lwd.ticks=0.5,lwd=0,pos=-0.27,hadj=1)


plot(1:25,result_robust_MCD$statistics_T2,type="l",ylab=expression(T^2),xlab="Observation",
     ylim=c(result_robust_MCD$LCL,result_robust_MCD$UCL),
     main="T Squared MCD Robust")
grid(nx=10,ny=10,lwd=1,lty=3,equilogs=F)
points(1:25,result_robust_MCD$statistics_T2,type="p",pch=20,cex=1.5)
abline(h=result_robust_MCD$UCL,lty=2,col="blue",lwd=2)
abline(h=result_robust_MCD$LCL,lty=2,col="blue",lwd=2)
mtext("UCL",side=4,at=c(25,result_robust_MCD$UCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
mtext("LCL",side=4,at=c(0,result_robust_MCD$LCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
points(index_robust_MCD_LCL,result_robust_MCD_LCL,col="red",type="p",pch=20)
points(index_robust_MCD_UCL,result_robust_MCD_UCL,col="red",type="p",pch=20)
xmintick=seq(1,25,by=1)
axis(1,at=xmintick,labels=FALSE,lwd.ticks=0.5,lwd=0,pos=-0.27,hadj=1)


plot(1:25,result_robust_MVE$statistics_T2,type="l",ylab=expression(T^2),xlab="Observation",
     ylim=c(result_robust_MVE$LCL,result_robust_MVE$UCL),
     main="T Squared MVE Robust")
grid(nx=10,ny=10,lwd=1,lty=3,equilogs=F)
points(1:25,result_robust_MVE$statistics_T2,type="p",pch=20,cex=1.5)
abline(h=result_robust_MVE$UCL,lty=2,col="blue",lwd=2)
abline(h=result_robust_MVE$LCL,lty=2,col="blue",lwd=2)
mtext("UCL",side=4,at=c(25,result_robust_MVE$UCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
mtext("LCL",side=4,at=c(0,result_robust_MVE$LCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
points(index_robust_MVE_LCL,result_robust_MVE_LCL,col="red",type="p",pch=20)
points(index_robust_MVE_UCL,result_robust_MVE_UCL,col="red",type="p",pch=20)
xmintick=seq(1,25,by=1)
axis(1,at=xmintick,labels=FALSE,lwd.ticks=0.5,lwd=0,pos=-0.27,hadj=1)
```



g) Generate another 25 future observations, using bivariate normal distribution with $\mu = (2.4, 6)$ and covariance matrix $\text{var}(x_1) = 1$; $\text{var}(x_2) = 0.5$, $\text{cov}(x_1, x_2) = 0.3$. and repeat (e).

Answer:

```
set.seed("6659")
mu_new<-c(2.4,6)
sigma_new<-matrix(c(1,0.3,0.3,0.5),2,2)
x_new<-mvrnorm(n_new,mu_new,sigma_new)
x_new
```

```
##            [,1]     [,2]
##  [1,] 2.4718998 5.940013
##  [2,] 1.6500771 5.684950
##  [3,] 3.4301669 6.474249
##  [4,] 2.8002897 6.362684
##  [5,] 3.1858740 7.358136
##  [6,] 2.3946885 5.090274
##  [7,] 1.6020456 5.616827
##  [8,] 2.7919774 7.082168
##  [9,] 1.1603203 6.303617
## [10,] 2.6544766 5.942188
## [11,] 4.6221453 5.862009
## [12,] 3.4203105 5.363667
## [13,] 3.8156500 7.242286
## [14,] 2.9740358 5.969157
## [15,] 1.3377744 5.260924
## [16,] 3.3555846 5.370191
```

```
## [17,] 1.6034503 6.336655
## [18,] 2.1091850 6.164499
## [19,] 0.4108093 5.251975
## [20,] 2.2103749 5.986103
## [21,] 2.4472102 7.083392
## [22,] 2.9799280 6.366280
## [23,] 1.9750477 6.509975
## [24,] 3.5704117 5.838395
## [25,] 2.6723340 6.797148
```

```r
#-------------------------------------
#         classic
#-------------------------------------
Mu_hat_Classic<-colMeans(x)
S_hat_Classic<-cov(x)
result_classic<-Stats_T2_func(x_new,Mu_hat_Classic,S_hat_Classic,25,1,2,0.95)
index_classic_UCL<-which(result_classic$statistics_T2>result_classic$UCL)
result_classic_UCL<-result_classic$statistics_T2[index_classic_UCL]
index_classic_LCL<-which(result_classic$statistics_T2<result_classic$LCL)
result_classic_LCL<-result_classic$statistics_T2[index_classic_LCL]
#-------------------------------------
#         Robust(MCD)
#-------------------------------------
estim_robust_MCD<-covMcd(x,cor=FALSE)
Mu_hat_Robust_MCD<-estim_robust_MCD$center
S_hat_Robust_MCD<-estim_robust_MCD$cov

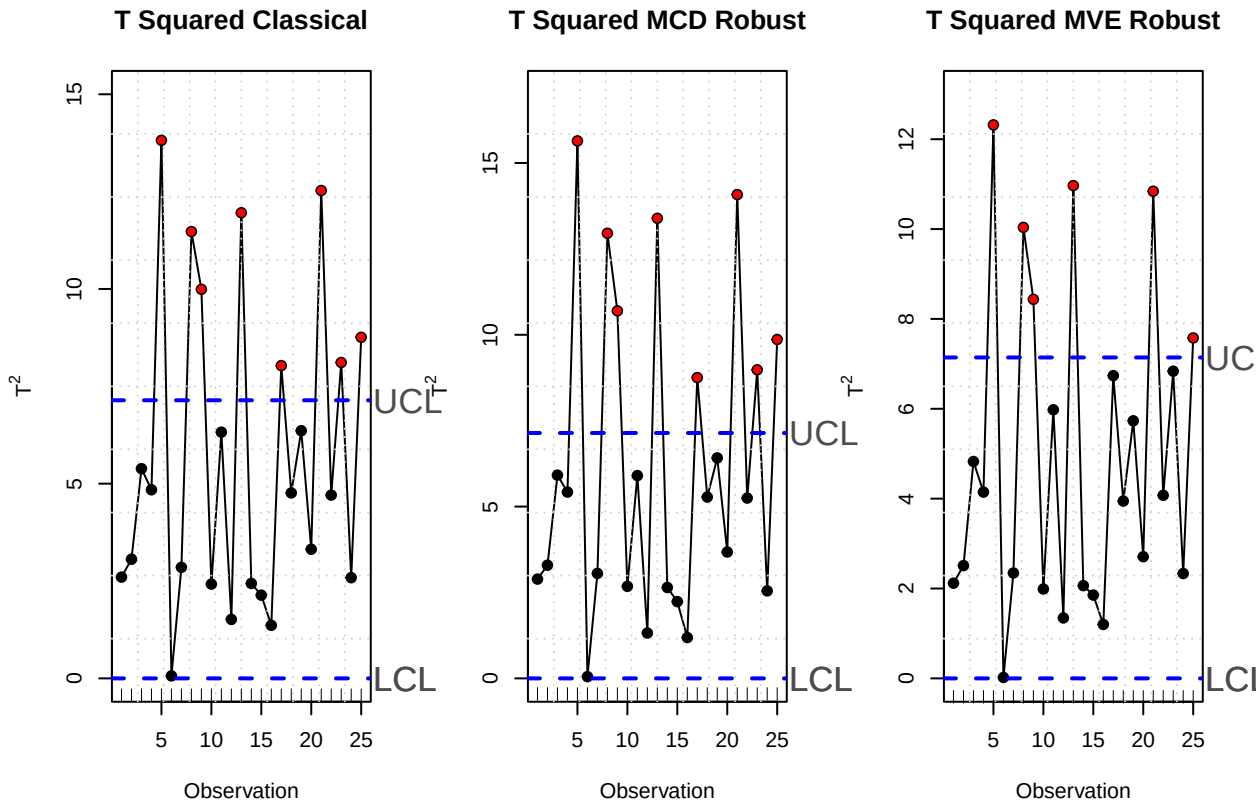
result_robust_MCD<-Stats_T2_func(x_new,Mu_hat_Robust_MCD,S_hat_Robust_MCD,25,1,2,0.95)
index_robust_MCD_UCL<-which(result_robust_MCD$statistics_T2>result_robust_MCD$UCL)
result_robust_MCD_UCL<-result_robust_MCD$statistics_T2[index_robust_MCD_UCL]
index_robust_MCD_LCL<-which(result_robust_MCD$statistics_T2<result_robust_MCD$LCL)
result_robust_MCD_LCL<-result_robust_MCD$statistics_T2[index_robust_MCD_LCL]
#-------------------------------------
#         Robust(MVE)
#-------------------------------------
estim_robust_MVE<-CovMve(x)
Mu_hat_Robust_MVE<-estim_robust_MVE$center
S_hat_Robust_MVE<-estim_robust_MVE$cov

result_robust_MVE<-Stats_T2_func(x_new,Mu_hat_Robust_MVE,S_hat_Robust_MVE,25,1,2,0.95)
index_robust_MVE_UCL<-which(result_robust_MVE$statistics_T2>result_robust_MVE$UCL)
result_robust_MVE_UCL<-result_robust_MVE$statistics_T2[index_robust_MVE_UCL]
index_robust_MVE_LCL<-which(result_robust_MVE$statistics_T2<result_robust_MVE$LCL)
result_robust_MVE_LCL<-result_robust_MVE$statistics_T2[index_robust_MVE_LCL]
#-------------------------------------------------------------------------------
par(mfrow=c(1,3))
plot(1:25,result_classic$statistics_T2,type="l",ylab=expression(T^2),xlab="Observation",
     ylim=c(result_classic$LCL,15),
     main="T Squared Classical")
grid(nx=10,ny=10,lwd=1,lty=3,equilogs=F)
points(1:25,result_classic$statistics_T2,type="p",pch=20,cex=1.5)
abline(h=result_classic$UCL,lty=2,col="blue",lwd=2)
abline(h=result_classic$LCL,lty=2,col="blue",lwd=2)
mtext("UCL",side=4,at=c(25,result_classic$UCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
```

```
mtext("LCL",side=4,at=c(0,result_classic$LCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
points(index_classic_LCL,result_classic_LCL,col="red",type="p",pch=20)
points(index_classic_UCL,result_classic_UCL,col="red",type="p",pch=20)
xmintick=seq(1,25,by=1)
axis(1,at=xmintick,labels=FALSE,lwd.ticks=0.5,lwd=0,pos=-0.27,hadj=1)

plot(1:25,result_robust_MCD$statistics_T2,type="l",ylab=expression(T^2),xlab="Observation",
     ylim=c(result_robust_MCD$LCL,17),
     main="T Squared MCD Robust")
grid(nx=10,ny=10,lwd=1,lty=3,equilogs=F)
points(1:25,result_robust_MCD$statistics_T2,type="p",pch=20,cex=1.5)
abline(h=result_robust_MCD$UCL,lty=2,col="blue",lwd=2)
abline(h=result_robust_MCD$LCL,lty=2,col="blue",lwd=2)
mtext("UCL",side=4,at=c(25,result_robust_MCD$UCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
mtext("LCL",side=4,at=c(0,result_robust_MCD$LCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
points(index_robust_MCD_LCL,result_robust_MCD_LCL,col="red",type="p",pch=20)
points(index_robust_MCD_UCL,result_robust_MCD_UCL,col="red",type="p",pch=20)
xmintick=seq(1,25,by=1)
axis(1,at=xmintick,labels=FALSE,lwd.ticks=0.5,lwd=0,pos=-0.27,hadj=1)

plot(1:25,result_robust_MVE$statistics_T2,type="l",ylab=expression(T^2),xlab="Observation",
     ylim=c(result_robust_MVE$LCL,13),
     main="T Squared MVE Robust")
grid(nx=10,ny=10,lwd=1,lty=3,equilogs=F)
points(1:25,result_robust_MVE$statistics_T2,type="p",pch=20,cex=1.5)
abline(h=result_robust_MVE$UCL,lty=2,col="blue",lwd=2)
abline(h=result_robust_MVE$LCL,lty=2,col="blue",lwd=2)
mtext("UCL",side=4,at=c(25,result_robust_MVE$UCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
mtext("LCL",side=4,at=c(0,result_robust_MVE$LCL),las=1,line=0.1,col=gray(0.3),cex=1.1)
points(index_robust_MVE_LCL,result_robust_MVE_LCL,col="red",type="p",pch=20)
points(index_robust_MVE_UCL,result_robust_MVE_UCL,col="red",type="p",pch=20)
xmintick=seq(1,25,by=1)
axis(1,at=xmintick,labels=FALSE,lwd.ticks=0.5,lwd=0,pos=-0.27,hadj=1)
```



f) Offer your comments. Compare your results with univariate charts for individual observations.
Answer:
We deduce from the T2 control charts based on different 25 future observations that:

1. The T^2 control chart generated using the classical estimators, the MCD Robust estimators and the MVE Robust estimators based on 25 future observations, using bivariate normal distribution with $\mu = (2, 5)$ shows that no mean vector were outside control limits.

2. The T^2 control chart generated using the classical estimators and the MCD Robust estimators based on 25 future observations, using bivariate normal distribution with $\mu = (2.4, 6)$ shows that eight points were outside control limits.

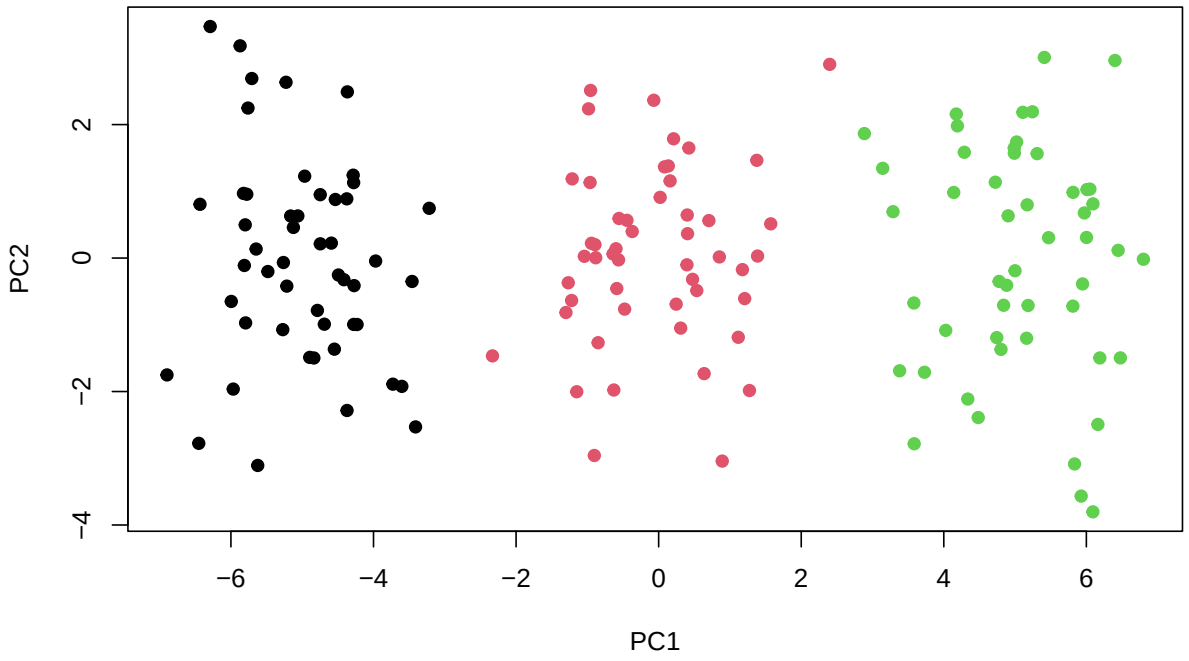
2. The T^2 control chart generated using the MVE Robust estimators based on 25 future observations, using bivariate normal distribution with $\mu = (2.4, 6)$ was similar and shows that six points were outside control limits.

Question 2.a) Generate a simulated data set with 50 observations in each of three classes (i.e. 150 observations total), and 50 variables. Be sure to add a mean shift to the observations in each class so that there are three distinct classes and choose covariance matrix of your choice.
Answer:

```
set.seed(1369)
X <- rbind(matrix(rnorm(50*50,mean=0),nrow=50),matrix(rnorm(50*50,mean=0.7),nrow=50),
           matrix(rnorm(50*50,mean=1.4),nrow=50))
```

b) Perform PCA on the 150 observations and plot the first two principal component score vectors. Use a different color to indicate the observations in each of the three classes. If the three classes appear separated in this plot, If not, then return to part (a) and modify the simulation so that there is greater separation between the three classes.
Answer:

```
PCA<-prcomp(X)
plot(PCA$x[,1:2],col=c(rep(1,50),rep(2,50),rep(3,50)),pch =19)
```



c) Use diagnostic tools such as variance contribution plot and offer your comments.
Answer:

```
suppressWarnings({library(factoextra)})
summary(PCA)
```

```
## Importance of components:
##                           PC1     PC2     PC3     PC4     PC5     PC6    PC7
## Standard deviation     4.2196 1.51853 1.45382 1.42124 1.38032 1.33951 1.3118
## Proportion of Variance 0.2732 0.03538 0.03243 0.03099 0.02923 0.02753 0.0264
## Cumulative Proportion  0.2732 0.30858 0.34101 0.37201 0.40124 0.42877 0.4552
##                           PC8     PC9    PC10    PC11   PC12    PC13    PC14
## Standard deviation     1.29766 1.27164 1.25464 1.21885 1.2163 1.19860 1.16497
```

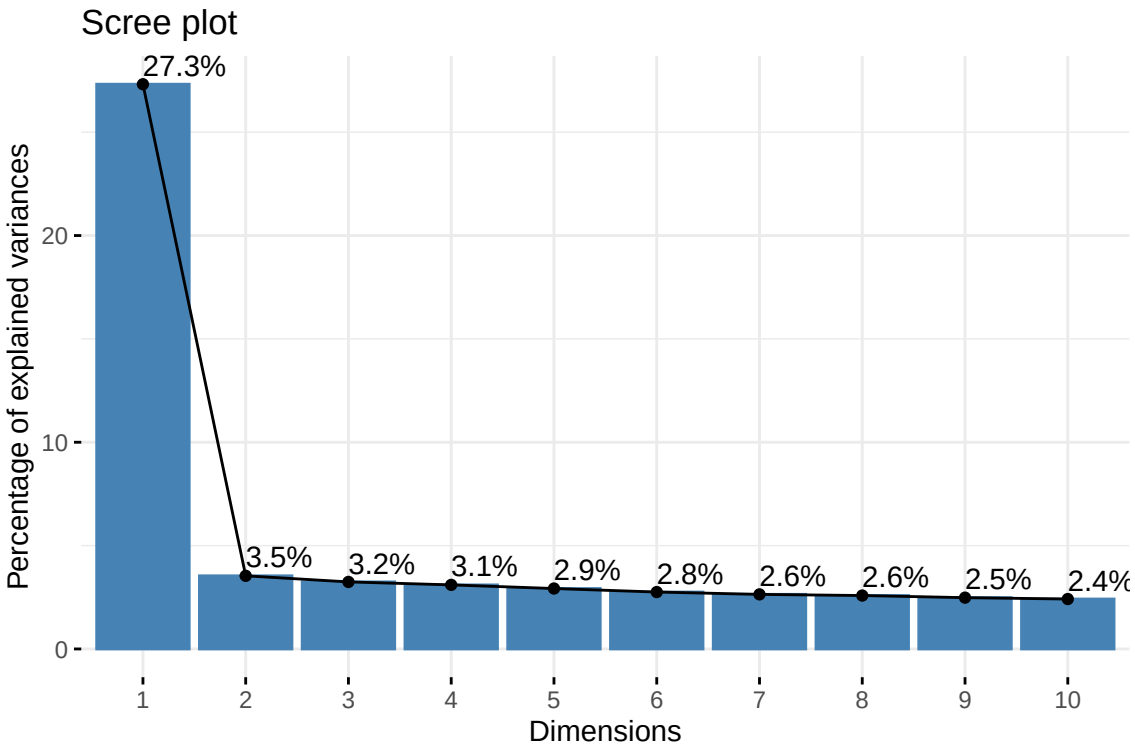
```
## Proportion of Variance 0.02584 0.02481 0.02415 0.02279 0.0227 0.02204 0.02082
## Cumulative Proportion  0.48102 0.50583 0.52998 0.55278 0.5755 0.59752 0.61834
##                           PC15    PC16    PC17    PC18    PC19   PC20    PC21
## Standard deviation     1.14559 1.11091 1.09951 1.07606 1.05383 1.0496 1.01662
## Proportion of Variance 0.02014 0.01894 0.01855 0.01777 0.01704 0.0169 0.01586
## Cumulative Proportion  0.63848 0.65742 0.67597 0.69373 0.71077 0.7277 0.74354
##                           PC22    PC23    PC24    PC25    PC26    PC27    PC28
## Standard deviation     1.00586 0.99102 0.97333 0.93882 0.92420 0.90693 0.89092
## Proportion of Variance 0.01552 0.01507 0.01454 0.01352 0.01311 0.01262 0.01218
## Cumulative Proportion  0.75906 0.77413 0.78867 0.80219 0.81529 0.82792 0.84009
##                           PC29    PC30    PC31    PC32    PC33    PC34    PC35
## Standard deviation     0.88667 0.86904 0.84481 0.82492 0.79703 0.79385 0.77190
## Proportion of Variance 0.01206 0.01159 0.01095 0.01044 0.00975 0.00967 0.00914
## Cumulative Proportion  0.85216 0.86375 0.87470 0.88514 0.89489 0.90455 0.91370
##                           PC36    PC37    PC38    PC39    PC40    PC41    PC42
## Standard deviation     0.74396 0.73637 0.71675 0.69356 0.67913 0.64975 0.64291
## Proportion of Variance 0.00849 0.00832 0.00788 0.00738 0.00708 0.00648 0.00634
## Cumulative Proportion  0.92219 0.93051 0.93839 0.94577 0.95285 0.95933 0.96567
##                           PC43    PC44    PC45    PC46    PC47    PC48    PC49
## Standard deviation     0.61328 0.59256 0.55434 0.54239 0.52602 0.49277 0.46132
## Proportion of Variance 0.00577 0.00539 0.00472 0.00451 0.00425 0.00373 0.00327
## Cumulative Proportion  0.97144 0.97683 0.98154 0.98606 0.99030 0.99403 0.99729
##                           PC50
## Standard deviation     0.41993
## Proportion of Variance 0.00271
## Cumulative Proportion  1.00000
```

```
eig.val<-get_eigenvalue(PCA)
eig.val
```

```
##        eigenvalue variance.percent cumulative.variance.percent
## Dim.1  17.8053956       27.3202552                    27.32026
## Dim.2   2.3059256        3.5381678                    30.85842
## Dim.3   2.1135794        3.2430354                    34.10146
## Dim.4   2.0199362        3.0993512                    37.20081
## Dim.5   1.9052730        2.9234141                    40.12422
## Dim.6   1.7942922        2.7531273                    42.87735
## Dim.7   1.7208059        2.6403714                    45.51772
## Dim.8   1.6839257        2.5837831                    48.10151
## Dim.9   1.6170714        2.4812031                    50.58271
## Dim.10  1.5741302        2.4153150                    52.99802
## Dim.11  1.4856053        2.2794841                    55.27751
## Dim.12  1.4794788        2.2700838                    57.54759
## Dim.13  1.4366427        2.2043569                    59.75195
## Dim.14  1.3571591        2.0823987                    61.83435
## Dim.15  1.3123670        2.0136706                    63.84802
## Dim.16  1.2341247        1.8936171                    65.74163
## Dim.17  1.2089294        1.8549580                    67.59659
## Dim.18  1.1579114        1.7766769                    69.37327
## Dim.19  1.1105517        1.7040091                    71.07728
## Dim.20  1.1017158        1.6904515                    72.76773
## Dim.21  1.0335101        1.5857980                    74.35353
## Dim.22  1.0117548        1.5524170                    75.90595
## Dim.23  0.9821203        1.5069464                    77.41289
## Dim.24  0.9473714        1.4536284                    78.86652
```

```
## Dim.25  0.8813868        1.3523828                80.21890
## Dim.26  0.8541392        1.3105748                81.52948
## Dim.27  0.8225270        1.2620695                82.79155
## Dim.28  0.7937360        1.2178932                84.00944
## Dim.29  0.7861919        1.2063176                85.21576
## Dim.30  0.7552319        1.1588132                86.37457
## Dim.31  0.7137056        1.0950961                87.46967
## Dim.32  0.6805007        1.0441471                88.51381
## Dim.33  0.6352498        0.9747150                89.48853
## Dim.34  0.6301979        0.9669635                90.45549
## Dim.35  0.5958228        0.9142189                91.36971
## Dim.36  0.5534730        0.8492383                92.21895
## Dim.37  0.5422423        0.8320061                93.05096
## Dim.38  0.5137331        0.7882621                93.83922
## Dim.39  0.4810201        0.7380680                94.57729
## Dim.40  0.4612240        0.7076932                95.28498
## Dim.41  0.4221807        0.6477860                95.93277
## Dim.42  0.4133322        0.6342089                96.56697
## Dim.43  0.3761161        0.5771053                97.14408
## Dim.44  0.3511272        0.5387629                97.68284
## Dim.45  0.3072914        0.4715020                98.15434
## Dim.46  0.2941830        0.4513888                98.60573
## Dim.47  0.2766980        0.4245600                99.03029
## Dim.48  0.2428261        0.3725876                99.40288
## Dim.49  0.2128166        0.3265417                99.72942
## Dim.50  0.1763429        0.2705772               100.00000
```

```
fviz_eig(PCA,col.var="blue",addlabel =TRUE)
```

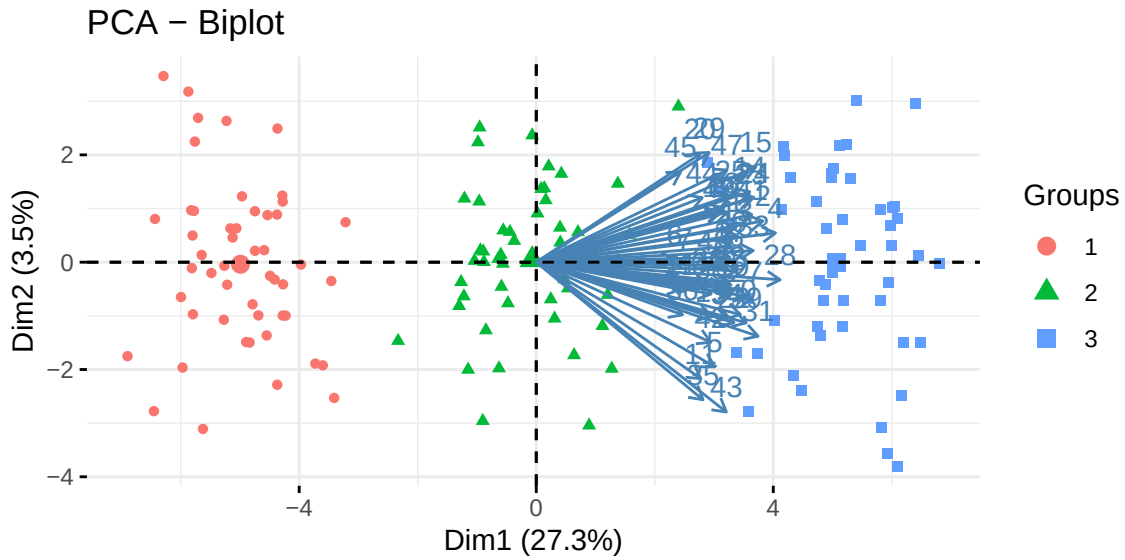


The first principal component has a much larger standard deviation than the other principal

components. The proportion of variance accounted for by PC1 is 0.27. we can see that from the first 34 PCs, over 90% of the variance in the data is explained.

d) Also use loading plots & scatter plots of for first two PCs and interpret the results
Answer:

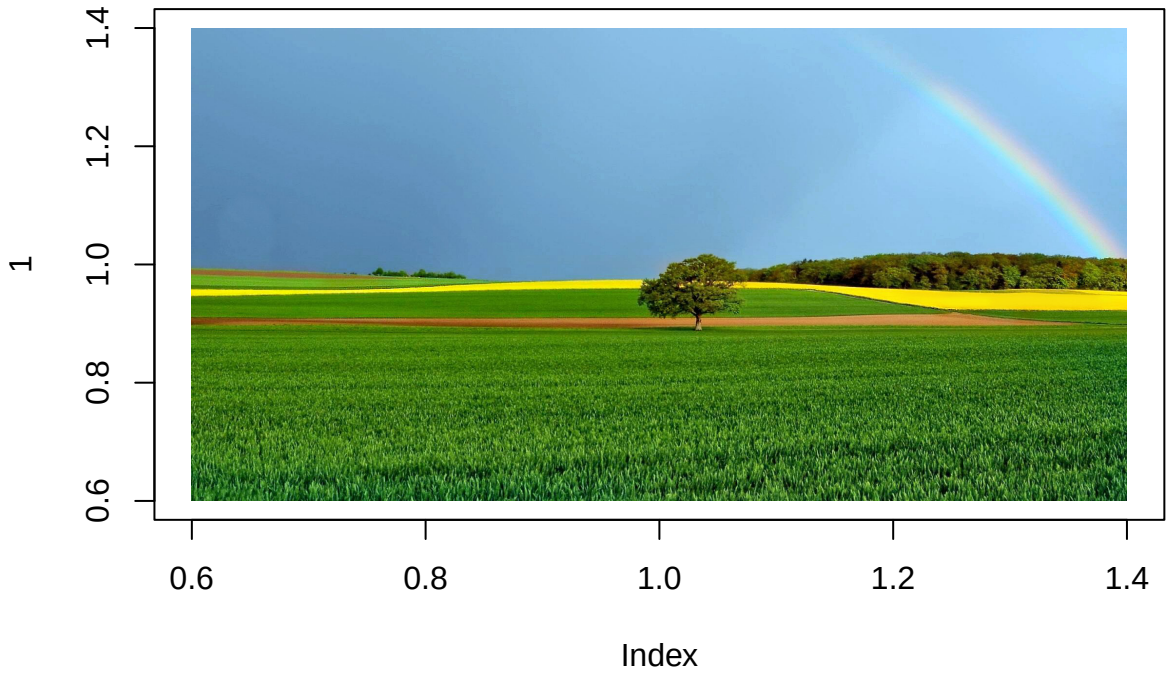
```
#PCA$rotation
fviz_pca_biplot(PCA,label="var",habillage=c(rep(1,50),rep(2,50),rep(3,50)))
```



As can be seen, this is a biplot that describes how each variable contributes to the two principal components of PCA Model. For instance, the first principal dimension (component), which is represented in the horizontal axis, has positive components for all 50 variables, which explains why all nine vectors appear on the right-hand side of the plot. The second principal component has positive coefficients for some variables and negative ones for some other.

Question 3. a) Choose any photo of your choice and convert it into the numerical data. Perform a PCA and reconstruct back the data based on first 100 PCA. Compare the file sizes of original photo and new photo based on PCA. Comment on the quality of photo based on PCA.
Answer:

```
suppressWarnings({library(jpeg)})
img <-readJPEG("D:/photos.jpg")
plot(1, type='n')
rasterImage(img, 0.6, 0.6, 1.4, 1.4)
```

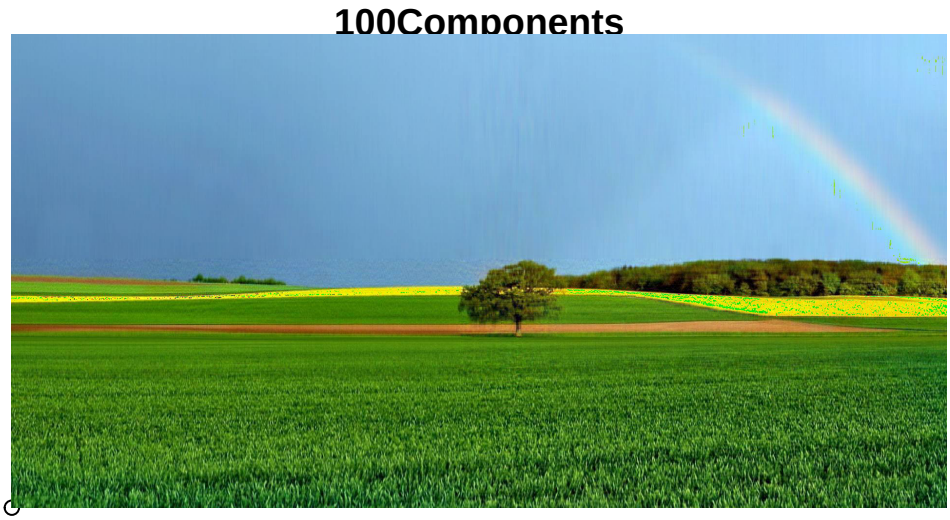



```
suppressWarnings({library(gridExtra)})
suppressWarnings({library(factoextra)})
suppressWarnings({library(abind)})

red<-img[,,1]
green<-img[,,2]
blue<-img[,,3]
red.pca<-prcomp(red,center=FALSE,scale.=FALSE)
green.pca<-prcomp(green,center=FALSE,scale.=FALSE)
blue.pca<-prcomp(blue,center=FALSE,scale.=FALSE)
list.img.pca<-list(red.pca,green.pca,blue.pca)

new_image<-abind(red.pca$x[,1:100] %*% t(red.pca$rotation[,1:100]),
                 green.pca$x[,1:100] %*% t(green.pca$rotation[,1:100]),
                 blue.pca$x[,1:100] %*% t(blue.pca$rotation[,1:100]),
                 along = 3)
writeJPEG(new_image, paste0("Compressed_image_with_",100,"_components.jpg"))
```

```
image_plot <- function(path, plot_name){
  require("jpeg")
  img <- readJPEG(path)
  d <- dim(img)
  plot(0,0,xlim=c(0,d[2]),ylim=c(0,d[2]),xaxt="n",yaxt="n",xlab="",ylab="",bty="n")
  title(plot_name, line = -0.5)
  rasterImage(img,0,0,d[2],d[2])
}
image_plot(paste0("Compressed_image_with_",100,"_components.jpg"),
             paste0(round(100,0),"Components"))
```



From the comparison of the original photo and new photo based on PCA, we can see that the quality of the images kept increasing as the number of principal components increased.

```
suppressWarnings({library(kableExtra)})
table <- matrix(0,2,3)
colnames(table)<-c("Number of components","Image size (kilobytes)","Saved Disk Space (kilobytes)")
table[,1]<- c(100,"Original Rock image")
table[2,2:3]<- round(c(file.info("D:/photos.jpg")$size/1024,0),2)
path <- paste0("Compressed_image_with_",table[1,1],"_components.jpg")
table[1,2]<-round(file.info(path)$size/1024,2)
table[1,3]<-round(as.numeric(table[2,2])-as.numeric(table[1,2]),2)
data_table<-knitr::kable(table,caption="", booktabs = T,label="kabletable")%>%
kable_styling(bootstrap_options=c("striped","hover","condensed","responsive"))
kable_classic_2(data_table,full_width=F,html_font="Cambria")
```

Table 1:

Number of components	Image size (kilobytes)	Saved Disk Space (kilobytes)
100	235.03	190.04
Original Rock image	425.07	0

Image compression with principal component analysis helped to save disk space of about 45% **with little or no loss in image quality. Not only has it saved disk space but it has made it easier and efficient to transmit these images between different sets of locations.**

b) i) For matrix completion, the codes used in the book "Statistical Learning using R" used the svd() function in R. Instead we can use the prcomp(0). write a function using prcomp() for matrix completion.

Answer:

```
mat_completion_pca_func<-function(X,M=1){
pr.out<-prcomp(X,scale=TRUE)
S<-X
d_pca<-pr.out$sdev*sqrt(max(1,nrow(S)-1))
v_pca<-pr.out$rotation
u_pca<-S%*%pr.out$rotation%*%solve(diag(d_pca))
result<-u_pca[,1:M,drop=FALSE]%*%
(d_pca[1:M]*t(v_pca[,1:M,drop=FALSE]))
round(result,7)
}
#-------------------------------------------
mat_completion_svd_func<-function(X,M=1){
svdob<-svd(X)
result<-with(svdob,u[,1:M,drop=FALSE] %*%
(d[1:M]*t(v[,1:M,drop=FALSE])))
round(result,7)
}
#-------------------------------------------
X <-data.matrix(scale(USArrests))
mat_completion_svd_func(X,M=1)
```

```
##             [,1]       [,2]       [,3]       [,4]
##  [1,]  0.5228559  0.5689892  0.2714198  0.5302052
##  [2,]  1.0345742  1.1258581  0.5370580  1.0491162
##  [3,]  0.9353819  1.0179137  0.4855663  0.9485297
##  [4,] -0.0750254 -0.0816451 -0.0389464 -0.0760799
##  [5,]  1.3390053  1.4571501  0.6950913  1.3578264
##  [6,]  0.8034959  0.8743910  0.4171029  0.8147899
##  [7,] -0.7207807 -0.7843775 -0.3741646 -0.7309120
##  [8,]  0.0253104  0.0275437  0.0131389  0.0256662
##  [9,]  1.5984593  1.7394966  0.8297765  1.6209273
## [10,]  0.8696616  0.9463947  0.4514502  0.8818856
## [11,] -0.4841769 -0.5268974 -0.2513411 -0.4909825
## [12,] -0.8699358 -0.9466931 -0.4515925 -0.8821637
## [13,]  0.7315306  0.7960760  0.3797450  0.7418130
## [14,] -0.2681540 -0.2918141 -0.1392015 -0.2719232
## [15,] -1.1955895 -1.3010802 -0.6206427 -1.2123947
## [16,] -0.4227561 -0.4600573 -0.2194570 -0.4286984
## [17,] -0.3983408 -0.4334877 -0.2067828 -0.4039399
```

```
## [18,]  0.8301569  0.9034044  0.4309429  0.8418256
## [19,] -1.2715502 -1.3837432 -0.6600747 -1.2894231
## [20,]  0.9354911  1.0180325  0.4856230  0.9486404
## [21,] -0.2579177 -0.2806747 -0.1338877 -0.2615430
## [22,]  1.1185563  1.2172502  0.5806540  1.1342788
## [23,] -0.8979904 -0.9772230 -0.4661560 -0.9106126
## [24,]  0.5286537  0.5752985  0.2744295  0.5360844
## [25,]  0.3696550  0.4022709  0.1918917  0.3748509
## [26,] -0.6288981 -0.6843879 -0.3264674 -0.6377379
## [27,] -0.6714372 -0.7306803 -0.3485499 -0.6808749
## [28,]  1.5249049  1.6594522  0.7915936  1.5463390
## [29,] -1.2646991 -1.3762876 -0.6565182 -1.2824757
## [30,]  0.0963233  0.1048222  0.0500024  0.0976772
## [31,]  1.0504292  1.1431120  0.5452885  1.0651940
## [32,]  0.8926299  0.9713895  0.4633733  0.9051767
## [33,]  0.5959674  0.6485516  0.3093728  0.6043444
## [34,] -1.5874158 -1.7274787 -0.8240437 -1.6097286
## [35,] -0.1198777 -0.1304549 -0.0622297 -0.1215627
## [36,] -0.1654050 -0.1799992 -0.0858634 -0.1677299
## [37,]  0.0313651  0.0341325  0.0162819  0.0318059
## [38,] -0.4713165 -0.5129023 -0.2446652 -0.4779413
## [39,] -0.4582427 -0.4986749 -0.2378784 -0.4646837
## [40,]  0.7006617  0.7624834  0.3637206  0.7105102
## [41,] -1.0545412 -1.1475868 -0.5474231 -1.0693639
## [42,]  0.5303764  0.5771732  0.2753238  0.5378314
## [43,]  0.7189190  0.7823516  0.3731982  0.7290241
## [44,] -0.2920823 -0.3178536 -0.1516229 -0.2961878
## [45,] -1.4861865 -1.6173176 -0.7714945 -1.5070764
## [46,] -0.0511070 -0.0556163 -0.0265301 -0.0518253
## [47,] -0.1150702 -0.1252232 -0.0597341 -0.1166876
## [48,] -1.1186328 -1.2173335 -0.5806937 -1.1343564
## [49,] -1.1033163 -1.2006655 -0.5727427 -1.1188245
## [50,] -0.3339193 -0.3633821 -0.1733409 -0.3386129
```

```
mat_completion_pca_func(X,M=1)
```

```
##                   Murder    Assault   UrbanPop       Rape
## Alabama         0.5228559  0.5689892  0.2714198  0.5302052
## Alaska          1.0345742  1.1258581  0.5370580  1.0491162
## Arizona         0.9353819  1.0179137  0.4855663  0.9485297
## Arkansas       -0.0750254 -0.0816451 -0.0389464 -0.0760799
## California      1.3390053  1.4571501  0.6950913  1.3578264
## Colorado        0.8034959  0.8743910  0.4171029  0.8147899
## Connecticut    -0.7207807 -0.7843775 -0.3741646 -0.7309120
## Delaware        0.0253104  0.0275437  0.0131389  0.0256662
## Florida         1.5984593  1.7394966  0.8297765  1.6209273
## Georgia         0.8696616  0.9463947  0.4514502  0.8818856
## Hawaii         -0.4841769 -0.5268974 -0.2513411 -0.4909825
## Idaho          -0.8699358 -0.9466931 -0.4515925 -0.8821637
## Illinois        0.7315306  0.7960760  0.3797450  0.7418130
## Indiana        -0.2681540 -0.2918141 -0.1392015 -0.2719232
## Iowa           -1.1955895 -1.3010802 -0.6206427 -1.2123947
## Kansas         -0.4227561 -0.4600573 -0.2194570 -0.4286984
## Kentucky       -0.3983408 -0.4334877 -0.2067828 -0.4039399
## Louisiana       0.8301569  0.9034044  0.4309429  0.8418256
```

```
## Maine          -1.2715502 -1.3837432 -0.6600747 -1.2894231
## Maryland        0.9354911  1.0180325  0.4856230  0.9486404
## Massachusetts  -0.2579177 -0.2806747 -0.1338877 -0.2615430
## Michigan        1.1185563  1.2172502  0.5806540  1.1342788
## Minnesota      -0.8979904 -0.9772230 -0.4661560 -0.9106126
## Mississippi     0.5286537  0.5752985  0.2744295  0.5360844
## Missouri        0.3696550  0.4022709  0.1918917  0.3748509
## Montana        -0.6288981 -0.6843879 -0.3264674 -0.6377379
## Nebraska       -0.6714372 -0.7306803 -0.3485499 -0.6808749
## Nevada          1.5249049  1.6594522  0.7915936  1.5463390
## New Hampshire  -1.2646991 -1.3762876 -0.6565182 -1.2824757
## New Jersey      0.0963233  0.1048222  0.0500024  0.0976772
## New Mexico      1.0504292  1.1431120  0.5452885  1.0651940
## New York        0.8926299  0.9713895  0.4633733  0.9051767
## North Carolina  0.5959674  0.6485516  0.3093728  0.6043444
## North Dakota   -1.5874158 -1.7274787 -0.8240437 -1.6097286
## Ohio           -0.1198777 -0.1304549 -0.0622297 -0.1215627
## Oklahoma       -0.1654050 -0.1799992 -0.0858634 -0.1677299
## Oregon          0.0313651  0.0341325  0.0162819  0.0318059
## Pennsylvania   -0.4713165 -0.5129023 -0.2446652 -0.4779413
## Rhode Island   -0.4582427 -0.4986749 -0.2378784 -0.4646837
## South Carolina  0.7006617  0.7624834  0.3637206  0.7105102
## South Dakota   -1.0545412 -1.1475868 -0.5474231 -1.0693639
## Tennessee       0.5303764  0.5771732  0.2753238  0.5378314
## Texas           0.7189190  0.7823516  0.3731982  0.7290241
## Utah           -0.2920823 -0.3178536 -0.1516229 -0.2961878
## Vermont        -1.4861865 -1.6173176 -0.7714945 -1.5070764
## Virginia       -0.0511070 -0.0556163 -0.0265301 -0.0518253
## Washington     -0.1150702 -0.1252232 -0.0597341 -0.1166876
## West Virginia  -1.1186328 -1.2173335 -0.5806937 -1.1343564
## Wisconsin      -1.1033163 -1.2006655 -0.5727427 -1.1188245
## Wyoming        -0.3339193 -0.3633821 -0.1733409 -0.3386129
```

b) ii) Use the generated data from Q.1(a). Consider only first 10 variables, so that your new data is of size 150×10. Randomly select 5 observations from this new data set and assume that these 5 observations are missing. Use Matrix Completion algorithm to estimate missing values using PCA. Offer your comments on the performance of the missing value estimation using PCA.

Answer:

```
set.seed(1369)
X<-rbind(matrix(rnorm(50*50, mean = 0), nrow = 50),
matrix(rnorm(50*50, mean=0.7), nrow = 50),
matrix(rnorm(50*50, mean=1.4), nrow = 50))
X<-scale(X)
#-----------------------------------------------
X2<-X[,1:10]
n_omit<-5
set.seed(15)
ina<-sample(seq(150),n_omit)
inb<-sample(1:10,n_omit,replace = TRUE)
index.na<-cbind(ina,inb)
X_NA<-X2
X_NA[index.na]<-NA
```

```
#------------------------------------------------
Xhat<-X_NA
x_bar<-colMeans(X_NA,na.rm=TRUE)
Xhat[index.na]<-x_bar[inb]
#------------------------------------------------
thresh<-1e-7
rel_err<-1
iter<-0
ismiss<-is.na(X_NA)
mssold<-mean((scale(X_NA,x_bar,FALSE)[!ismiss])^2)
mss0<-mean(X_NA[!ismiss]^2)
#------------------------------------------------
#                PCA Method
#------------------------------------------------
while(rel_err>thresh){
iter<-iter+1
Xapp<-mat_completion_pca_func(Xhat,M=1)
Xhat[ismiss]<-Xapp[ismiss]
mss<-mean(((X_NA-Xapp)[!ismiss])^2)
rel_err<-(mssold-mss)/mss0
mssold<-mss
cat("Iter:",iter,"MSS:",mss,
"Rel.Err:",rel_err,"\n")
}
```

```
## Iter: 1 MSS: 0.6442796 Rel.Err: 0.3507579
## Iter: 2 MSS: 0.6441895 Rel.Err: 9.080855e-05
## Iter: 3 MSS: 0.6441872 Rel.Err: 2.256003e-06
## Iter: 4 MSS: 0.6441871 Rel.Err: 1.396917e-07
## Iter: 5 MSS: 0.6441871 Rel.Err: 1.549141e-08
```

```
cor(Xapp[index.na],X[index.na])
```

```
## [1] 0.475557
```

```
Result<-rbind(X2[index.na],Xhat[ismiss])
rownames(Result)<-c("Exact value","Estimate value")
Result
```

```
##                      [,1]      [,2]       [,3]       [,4]      [,5]
## Exact value    -1.501188 0.7182161 -1.7587227 -0.3589292 0.6118685
## Estimate value -0.049260 0.0473933 -0.6758519  0.7866244 0.3289848
```

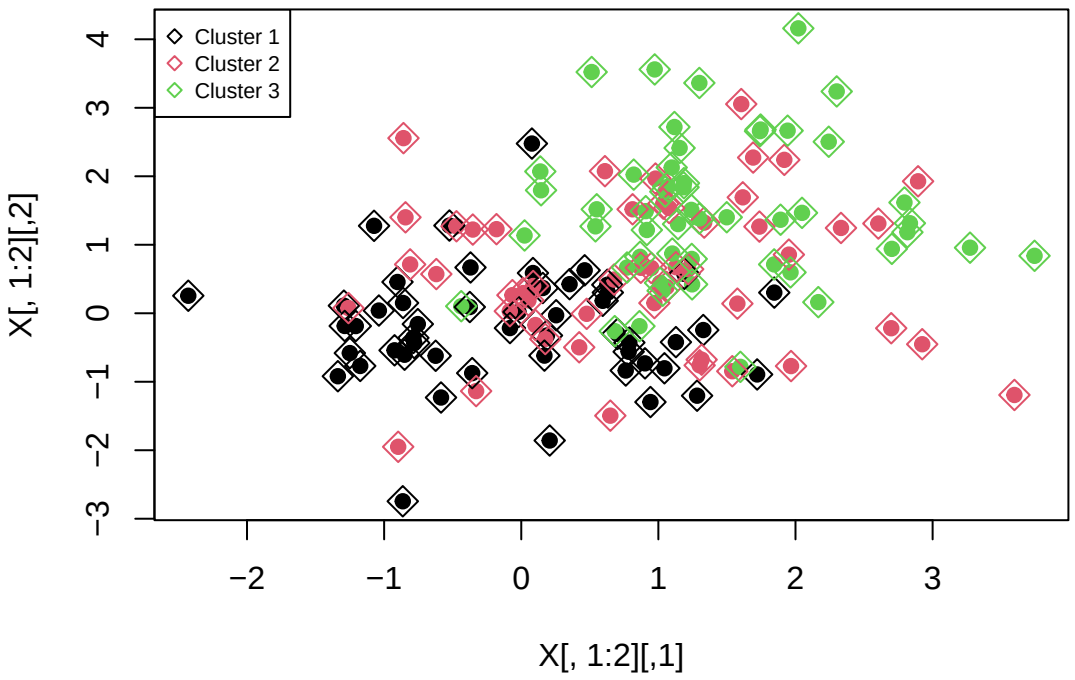
We see that after five iterations, the relative error has fallen below $\text{thresh} = 1e^{-7}$**, and so the algorithm terminates. When this happens, the mean squared error of the non-missing elements equals** 0.64419**.**

Question 4. For this problem, use the data in Q2(a).
a) Perform K-means clustering of the observations with $K = 3$. How well do the clusters that you obtained in K-means clustering compare to the true class labels? Hint: You can use the table() function in R to compare the true class labels to the class labels obtained by clustering. Be careful how you interpret the results: K-means clustering will arbitrarily number the clusters, so you cannot simply check whether the true class labels and clustering labels are the same.
Answer:

```
set.seed(1369)
X <- rbind(matrix(rnorm(50*50, mean = 0), nrow = 50),
matrix(rnorm(50*50, mean=0.7), nrow = 50),
matrix(rnorm(50*50, mean=1.4), nrow = 50))
#-----------------------------------------------------
res<-kmeans(X,centers=3,nstart=50)
true_class<-c(rep(1,50),rep(2,50),rep(3,50))
table(res$cluster,true_class,dnn=c("Clusters","Class Labels"))
```

```
##          Class Labels
## Clusters  1  2  3
##        1 50  0  0
##        2  0 50  0
##        3  0  0 50
```

```
plot(X[,1:2],col=c(rep(1,50),rep(2,50),rep(3,50)),pch =19)
points(X[res$cluster==1,1:2],pch=5,cex=1.5,col=1)
points(X[res$cluster==2,1:2],pch=5,cex=1.5,col=2)
points(X[res$cluster==3,1:2],pch=5,cex=1.5,col=3)
legend("topleft",c(paste("Cluster",sort(unique(res$cluster)))),
    pch=c(5,5,5),col=c(1,2,3),cex=.7)
```



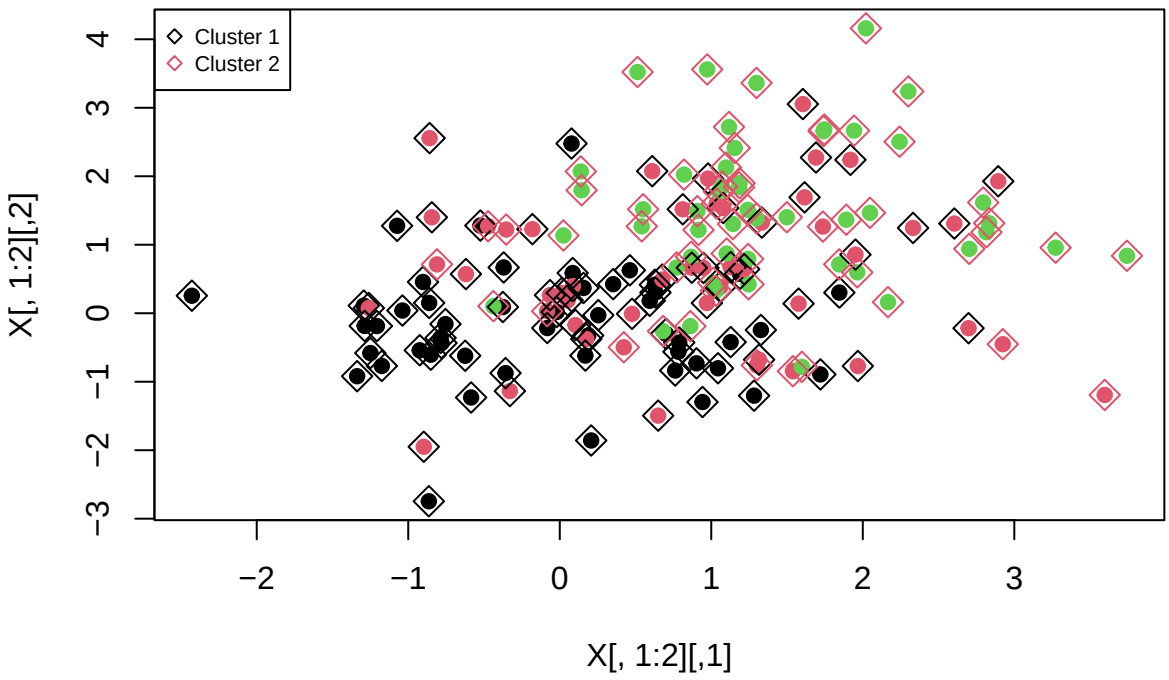
The observations are perfectly clustered.

b) Perform K-means clustering with $K = 2$. Describe your results.
Answer:

```
res<-kmeans(X,centers=2,nstart=50)
true_class<-c(rep(1,50),rep(2,50),rep(3,50))
table(res$cluster,true_class,dnn=c("Clusters","Class Labels"))
```

```
##          Class Labels
## Clusters  1  2  3
##        1 50 39  0
##        2  0 11 50
```

```
plot(X[,1:2],col=c(rep(1,50),rep(2,50),rep(3,50)),pch =19)
points(X[res$cluster==1,1:2],pch=5,cex=1.5,col=1)
points(X[res$cluster==2,1:2],pch=5,cex=1.5,col=2)
legend("topleft",c(paste("Cluster",sort(unique(res$cluster)))),
    pch=c(5,5),col=c(1,2),cex=.7)
```



In the above table, we see that two of the classes are perfectly assigned to two clusters. ie. class 1 to cluster 1 and class 3 to cluster 2. However as we only have two classes K-means has forced the class 2 to be split among the two clusters - however, majority of class2 observations go to cluster 1.

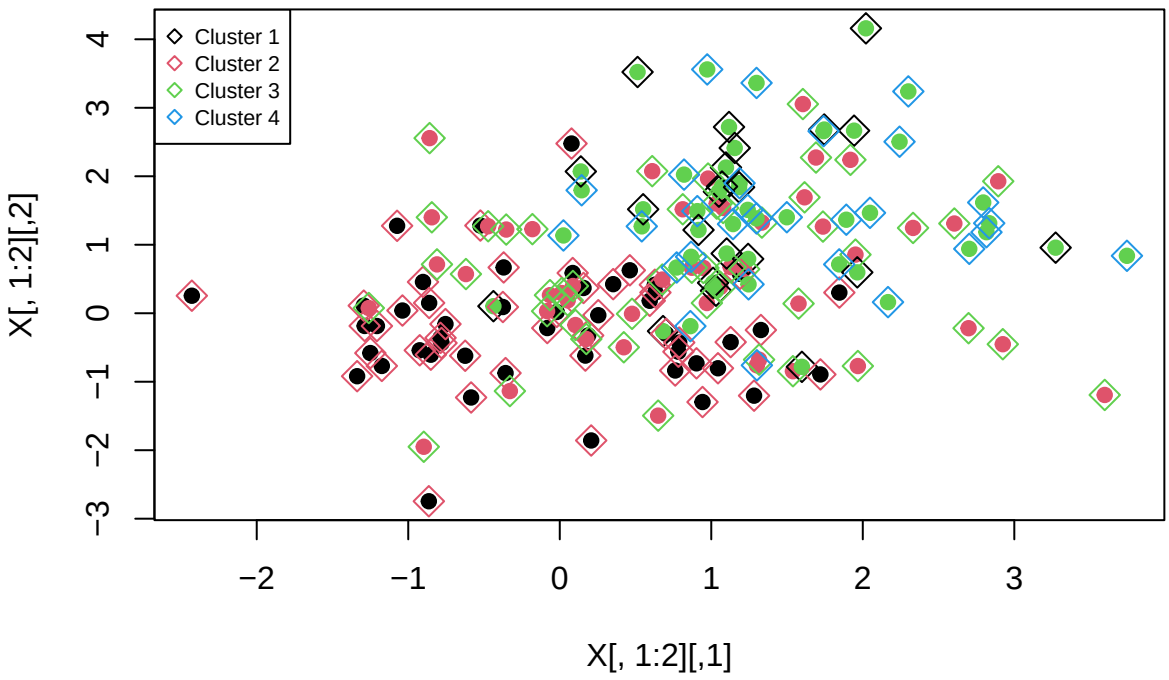
c) Now perform K-means clustering with $K = 4$, and describe your results.

Answer:

```
res<-kmeans(X,centers=4,nstart=50)
true_class<-c(rep(1,50),rep(2,50),rep(3,50))
table(res$cluster,true_class,dnn=c("Clusters","Class Labels"))
```

```
##          Class Labels
## Clusters  1  2  3
##        1  0  0 22
##        2 50  0  0
##        3  0 49  0
##        4  0  1 28
```

```
plot(X[,1:2],col=c(rep(1,50),rep(2,50),rep(3,50)),pch =19)
points(X[res$cluster==1,1:2],pch=5,cex=1.5,col=1)
points(X[res$cluster==2,1:2],pch=5,cex=1.5,col=2)
points(X[res$cluster==3,1:2],pch=5,cex=1.5,col=3)
points(X[res$cluster==4,1:2],pch=5,cex=1.5,col=4)
legend("topleft",c(paste("Cluster",sort(unique(res$cluster)))),
    pch=c(5,5,5,5),col=c(1,2,3,4),cex=.7)
```



In the above table, we see that class 1 is assigned entirely to Cluster 2. Majority of class 2 is assigned to Cluster 3. However k-means seemed to have forced some observations from Class 3 into a Cluster 1. Majority of class 2 is assigned to cluster 3. However k-means seemed to have forced some observations from class 2 into the fourth cluster.

d) Now perform K-means clustering with $K = 3$ on the first two principal component score vectors, rather than on the raw data. That is, perform K-means clustering on the 150×2 matrix of which the first column is the first principal component score vector, and the second column is the second principal component score vector. Comment on the results.

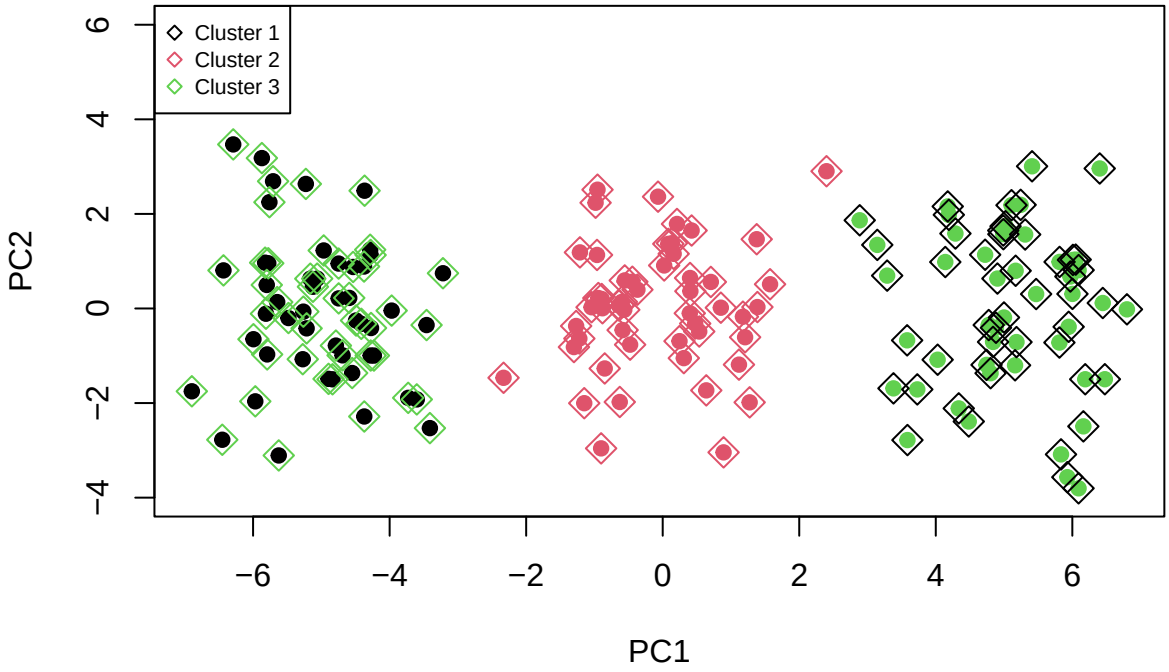
Answer:

```
PCA<-prcomp(X)
X.pca<-PCA$x
res<-kmeans(X.pca[,1:2],centers=3,nstart=50)
true_class<-c(rep(1,50),rep(2,50),rep(3,50))
table(res$cluster,true_class,dnn=c("Clusters","Class Labels"))
```

```
##          Class Labels
## Clusters  1  2  3
##        1  0  0 50
##        2  0 50  0
##        3 50  0  0
```

```
plot(X.pca[,1:2],ylim=c(-4,6),col=c(rep(1,50),rep(2,50),rep(3,50)),pch =19)
points(X.pca[res$cluster==1,1:2],pch=5,cex=1.5,col=1)
points(X.pca[res$cluster==2,1:2],pch=5,cex=1.5,col=2)
points(X.pca[res$cluster==3,1:2],pch=5,cex=1.5,col=3)
```

```
legend("topleft",c(paste("Cluster",sort(unique(res$cluster)))),
    pch=c(5,5,5),col=c(1,2,3),cex=.7)
```



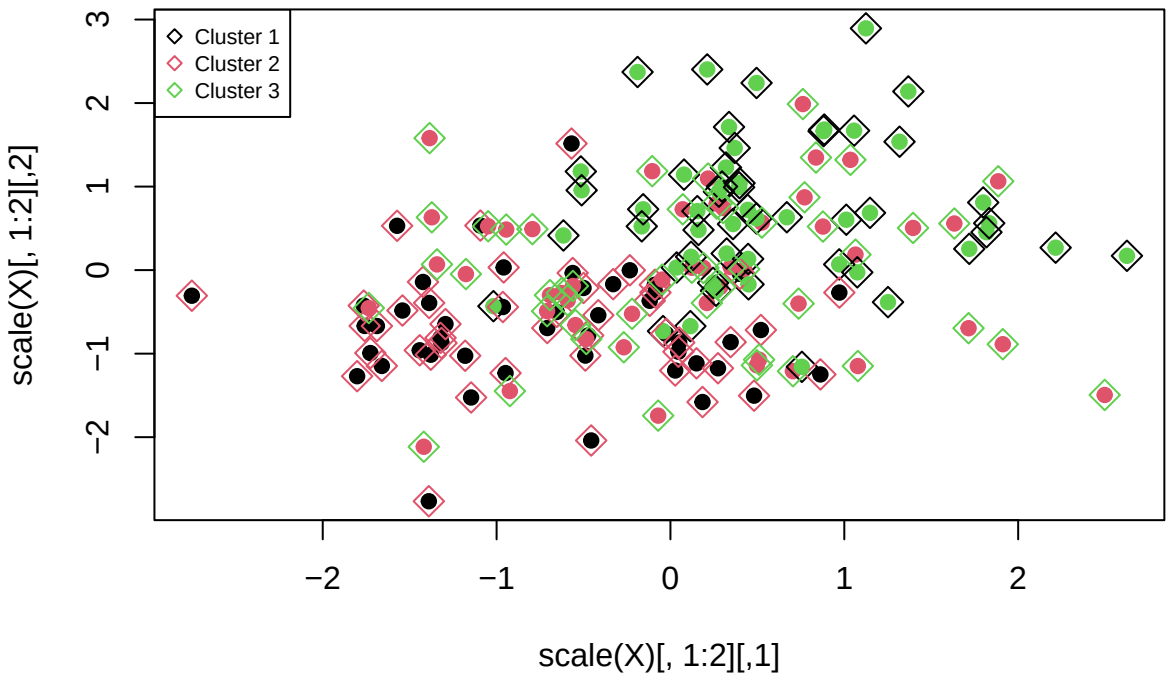
In the above table, it can be seen that only the first two principal components perfectly separate each class into unique clusters. Here we have reduced the variables from 50 to 2, and achieved almost the same accuracy as seen in part (a) where three clusters were also used.

g) Using the scale() function, perform K-means clustering with $K = 3$ on the data after scaling each variable to have standard deviation one. How do these results compare to those obtained in Q2? Explain.
Answer:

```
res<-kmeans(scale(X),centers=3,nstart=50)
true_class<-c(rep(1,50),rep(2,50),rep(3,50))
table(res$cluster,true_class,dnn=c("Clusters","Class Labels"))
```

```
##          Class Labels
## Clusters  1  2  3
##        1  0  0 50
##        2 50  0  0
##        3  0 50  0
```

```
plot(scale(X)[,1:2],col=c(rep(1,50),rep(2,50),rep(3,50)),pch =19)
points(scale(X)[res$cluster==1,1:2],pch=5,cex=1.5,col=1)
points(scale(X)[res$cluster==2,1:2],pch=5,cex=1.5,col=2)
points(scale(X)[res$cluster==3,1:2],pch=5,cex=1.5,col=3)
legend("topleft",c(paste("Cluster",sort(unique(res$cluster)))),
    pch=c(5,5,5),col=c(1,2,3),cex=.7)
```



In the above table, we see that each arbitrary cluster is assigned to one class only. Since scaling affects the distance between the observations, So, scaling does not change the results.